

TOTAL MODULES	CURRICULUM PART	CORE MECHANICS	REFERENCE STANDARD
72 Modules + 6 Practice Labs	Part 29 — Quality Assurance & Testing	Unit, Integration, E2E, Mocks & TDD	Jest / Vitest / Playwright 2026

🔧 Testing JavaScript — নির্ভরযোগ্যতা, বাগ প্রতিরোধ ও আত্মবিশ্বাসী রিফ্যাক্টরিং

প্রফেশনাল সফটওয়্যার ডেভেলপমেন্টে কোড শুধু কাজ করলেই যথেষ্ট নয়—ভবিষ্যতে নতুন ফিচার যুক্ত করার পর পূর্বের কোনো কার্যকারিতা ভেঙে পড়ল কিনা (Regression) তা স্বয়ংক্রিয়ভাবে নিশ্চিত করাই টেস্টিংয়ের লক্ষ্য। এই চ্যাপ্টারে আমরা **Unit Testing, Integration Testing, E2E Testing, Testing Pyramid, Jest/Vitest Assertions, Setup & Teardown Hooks, Async Testing, Mocking, Code Coverage, Test-Driven Development (TDD)** এবং CI/CD টেস্ট অটোমেশন পাইপলাইন শিখব।

🔥 1. Testing Fundamentals	🔥 2. Testing Pyramid	🔥 3. Jest & Assertions	🔥 4. Matchers & Suites	🔥 5. Async Testing
🔥 6. Mocking & Spies	🔥 7. Code Coverage	🔥 8. TDD Red-Green-Refactor	🔥 9. E2E with Playwright	🔥 10. CI/CD Pipeline

Part 01 — Testing Fundamentals, Motivations & Automated Testing (35.1 – 35.5)

35.1 Testing কী?

Testing হলো code-এর expected behavior এবং actual behavior compare করে দেখা। সহজভাবে:

💡 “আমি যে output আশা করছি, আমার program কি সত্যিই সেই output দিচ্ছে?”

♦ Real-Life Example

ধরো তুমি একটা calculator বানিয়েছ। তুমি আশা করছ:

🔧 TESTING ARCHITECTURE & EXECUTION FLOW

2 + 3 = 5

Testing করে আমরা যাচাই করব:

JAVASCRIPT SOURCEJAVASCRIPT

add(2, 3) === 5;

যদি 5 আসে → Test Pass ✅ অন্য কিছু আসে → Test Fail ❌

35.2 কেন Testing দরকার?

ধরো তোমার একটি বড় e-commerce website আছে। এখানে:

TESTING ARCHITECTURE & EXECUTION FLOW

```
Login
↓
Product Search
↓
Add to Cart
↓
Checkout
↓
Payment
↓
Order Confirmation
```

তুমি Login feature পরিবর্তন করলে ভুল করে Cart-এর code নষ্ট হয়ে যেতে পারে। Testing থাকলে automatically ধরা যেতে পারে:

TESTING ARCHITECTURE & EXECUTION FLOW

Login Test	✓
Search Test	✓
Cart Test	✗
Payment Test	✓

তখন developer বুঝতে পারবে কোথায় সমস্যা হয়েছে।

35.3 Testing-এর প্রধান উদ্দেশ্য

উদ্দেশ্য	অর্থ
Bug Detection	Bug খুঁজে বের করা
Reliability	Application reliable রাখা
Regression Prevention	পুরোনো feature নষ্ট হয়েছে কিনা দেখা
Refactoring Safety	Code পরিবর্তন করেও functionality ঠিক রাখা
Documentation	Code কীভাবে behave করার কথা তা বোঝানো
Quality	Overall software quality বৃদ্ধি

35.4 Manual Testing vs Automated Testing

Manual Testing

Developer/User নিজে application ব্যবহার করে পরীক্ষা করে। উদাহরণ:

TESTING ARCHITECTURE & EXECUTION FLOW

```
Open website
↓
Enter email
↓
Enter password
↓
Click Login
↓
Check result
```

♦ সুবিধা

- সহজ
- UI/UX যাচাই করা যায়
- Human behavior বোঝা যায়

♦ অসুবিধা

- সময় বেশি লাগে
- বারবার করতে হয়
- Human error হতে পারে

35.5 Automated Testing

Code নিজেই অন্য code-এর behavior পরীক্ষা করে। উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(add(2, 3)).toBe(5);
```

এখানে মানুষকে manually calculator চালাতে হচ্ছে না। Test framework automatically check করবে।

Part 02 — Testing Pyramid & Test Types (Unit, Integration, E2E, Regression) (35.6 – 35.12)

35.6 Testing-এর Levels

JavaScript application-এ সাধারণত কয়েক ধরনের testing দেখা যায়:

TESTING ARCHITECTURE & EXECUTION FLOW

Testing

- Unit Testing
- Integration Testing
- End-to-End Testing
- Component Testing
- Regression Testing

35.7 Unit Testing

Unit Test হলো application-এর সবচেয়ে ছোট testable অংশ পরীক্ষা করা। যেমন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}
```

এখন আমরা শুধু add() function-টি test করব।

TESTING ARCHITECTURE & EXECUTION FLOW

Input → add() → Output

♦ Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}
```

Test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(add(2, 3)).toBe(5);
```

এখানে পুরো application test করা হচ্ছে না। শুধু:

TESTING ARCHITECTURE & EXECUTION FLOW

add()

test করা হচ্ছে।

♦ Real-Life Example

একটি restaurant-এর kitchen-এ:

TESTING ARCHITECTURE & EXECUTION FLOW

Rice cooking → আলাদা test
Chicken cooking → আলাদা test
Sauce → আলাদা test

প্রতিটি ছোট অংশ আলাদাভাবে ঠিক আছে কিনা পরীক্ষা করা = Unit Testing-এর মতো।

35.8 Integration Testing

Integration Testing-এ একাধিক component/module একসাথে কাজ করছে কিনা test করা হয়। উদাহরণ:

TESTING ARCHITECTURE & EXECUTION FLOW

```
Login Function
  ↓
Authentication Service
  ↓
Database
```

শুধু Login function নয়। পুরো interaction test করা হচ্ছে।

♦ Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
loginUser()
  ↓
API
  ↓
Database
  ↓
User Response
```

এগুলো একসাথে ঠিকভাবে কাজ করছে কিনা পরীক্ষা করা হবে।

35.9 Unit vs Integration

Unit Testing	Integration Testing
ছোট অংশ test	একাধিক অংশ test
সাধারণত isolated	Components interact করে
দ্রুত	তুলনামূলক ধীর
Function test	API + DB ইত্যাদি test
সহজ debugging	Debugging তুলনামূলক কঠিন

35.10 End-to-End Testing

E2E Testing পুরো application-এর user journey পরীক্ষা করে। ধরো একটি shopping website:

TESTING ARCHITECTURE & EXECUTION FLOW

```
Open Website
  ↓
Login
  ↓
Search Product
  ↓
Open Product
  ↓
Add Cart
  ↓
Checkout
  ↓
Payment
  ↓
Order Complete
```

পুরো flow test করা হচ্ছে। এটাই End-to-End Testing।

35.11 Component Testing

বিশেষ করে React application-এ Component Testing গুরুত্বপূর্ণ। ধরো:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
<Button />
```

Test করা হচ্ছে:

- Button render হচ্ছে?
- Text ঠিক আছে?
- Click করলে function call হচ্ছে?
- Disabled হলে কাজ করছে না?

35.12 Regression Testing

ধরো তোমার application আগে ঠিকভাবে কাজ করছিল। তুমি নতুন feature যোগ করলে:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
New Feature  
↓  
Old Feature  
↓  
Broken ❌
```

নতুন change-এর কারণে পুরোনো functionality নষ্ট হয়েছে কিনা check করা হলো **Regression Testing**।

Part 03 — Test Cases, Scenarios, Assertions & Jest Setup (35.13 – 35.18)

35.13 Test Case কী?

একটি নির্দিষ্ট behavior পরীক্ষা করার জন্য যে condition এবং expected result তৈরি করা হয় সেটিই Test Case। উদাহরণ:

♦ Login Test Case

 TESTING ARCHITECTURE & EXECUTION FLOW

```
Test:  
Valid email + valid password  
  
Expected:  
Login successful
```

আরেকটি:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
Test:  
Valid email + wrong password  
  
Expected:  
❌ message
```

35.14 Test Scenario vs Test Case

◆ Test Scenario

High-level requirement।

🔧 TESTING ARCHITECTURE & EXECUTION FLOW

Test Login functionality

◆ Test Case

Specific condition।

🔧 TESTING ARCHITECTURE & EXECUTION FLOW

Valid email + valid password
Valid email + invalid password
Empty email
Empty password
Invalid email format

35.15 Assertion কী?

Testing-এর সবচেয়ে গুরুত্বপূর্ণ concept-এর একটি হলো **Assertion**। Assertion হলো:

💡 “আমি যে result আশা করছি, actual result কি সেটার সাথে match করছে?”

উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(2 + 3).toBe(5);
```

এখানে:

🔧 TESTING ARCHITECTURE & EXECUTION FLOW

Expected → 5
Actual → 5

তাই:

🔧 TESTING ARCHITECTURE & EXECUTION FLOW

PASS ✅

যদি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(2 + 3).toBe(6);
```

তাহলে:

🔧 TESTING ARCHITECTURE & EXECUTION FLOW

Expected → 6
Actual → 5

FAIL ❌

35.16 Jest কী?

JavaScript testing-এর জন্য বহুল ব্যবহৃত একটি testing framework হলো **Jest**। Jest দিয়ে আমরা:

- Unit Testing
- Integration Testing
- Mocking
- Assertions
- Coverage

ইত্যাদি করতে পারি।

35.17 Jest Install

Project তৈরি:

TERMINAL TEST COMMAND

BASH

```
-weight:700;">npm -weight:600;">init -y
```

Jest install:

TERMINAL TEST COMMAND

BASH

```
-weight:700;">npm -weight:600;">install --save-dev -weight:700;">jest
```

35.18 Simple Jest Test

ধরো:

♦ math.js

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}  
  
module.exports = { add };
```

Test file:

♦ math.test.js

JAVASCRIPT SOURCE

JAVASCRIPT

```
const { add } = require("./math");  
  
test("2 + 3 should equal 5", () => {  
  expect(add(2, 3)).toBe(5);  
});
```

Run:

TERMINAL TEST COMMAND

BASH

```
-weight:700;">npx -weight:700;">jest
```

Expected result:

🔪 TESTING ARCHITECTURE & EXECUTION FLOW

```
PASS ./math.test.js  
✓ 2 + 3 should equal 5
```

35.19 test() কী?

Jest-এর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test();
```

একটি test define করে। Syntax:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("test description", () => {
  // test code
});
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("addition works", () => {
  expect(2 + 2).toBe(4);
});
```

35.20 expect() কী?

expect() actual value-এর ওপর assertion করার জন্য ব্যবহার করা হয়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(value);
```

তারপর matcher:

JAVASCRIPT SOURCE

JAVASCRIPT

```
.toBe()
.toEqual()
.toContain()
.toBeTruthy()
.toBeFalsy()
```

ইত্যাদি।

35.21 toBe()

Primitive value comparison-এর জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(5).toBe(5);
```

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect("Hello").toBe("Hello");
```

35.22 toEqual()

Objects বা Arrays-এর content compare করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect({ name: "Ripon" }).toEqual({ name: "Ripon" });
```

Array:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect([1, 2, 3]).toEqual([1, 2, 3]);
```

◆ গুরুত্বপূর্ণ

JAVASCRIPT SOURCE

JAVASCRIPT

```
toBe();
```

আর

JAVASCRIPT SOURCE

JAVASCRIPT

```
toEqual();
```

এক জিনিস নয়।

35.23 Truthy / Falsy Testing

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(true).toBeTruthy();
```

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(false).toBeFalsy();
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function isLoggedIn(user) {  
  return Boolean(user);  
}
```

Test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(isLoggedIn({ name: "Ripon" })).toBeTruthy();
```

35.24 toContain()

Array বা string-এর মধ্যে কিছু আছে কিনা:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(["HTML", "CSS", "JavaScript"]).toContain("JavaScript");
```

String:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect("Hello World").toContain("World");
```

35.25 toBeNull()

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(null).toBeNull();
```

35.26 toBeUndefined()

JAVASCRIPT SOURCE

JAVASCRIPT

```
let value;

expect(value).toBeUndefined();
```

35.27 toBeDefined()

JAVASCRIPT SOURCE

JAVASCRIPT

```
const name = "Ripon";

expect(name).toBeDefined();
```

35.28 Multiple Tests

JAVASCRIPT SOURCE

JAVASCRIPT

```
function multiply(a, b) {
  return a * b;
}
```

Test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("2 × 3 = 6", () => {
  expect(multiply(2, 3)).toBe(6);
});

test("5 × 4 = 20", () => {
  expect(multiply(5, 4)).toBe(20);
});

test("10 × 0 = 0", () => {
  expect(multiply(10, 0)).toBe(0);
});
```

Part 05 — Test Suites (describe) & Lifecycle Hooks (Setup & Teardown) (35.29 – 35.34)

35.29 describe()

Related tests group করার জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
describe("Math Tests", () => {
  test("addition", () => {
    expect(2 + 3).toBe(5);
  });

  test("multiplication", () => {
    expect(2 * 3).toBe(6);
  });
});
```

Structure:

🔪 TESTING ARCHITECTURE & EXECUTION FLOW

```
Math Tests
├── addition
└── multiplication
```

35.30 Setup & Teardown

কোনো test-এর আগে বা পরে কিছু কাজ করতে হয়। যেমন:

TESTING ARCHITECTURE & EXECUTION FLOW

```
Connect database
↓
Run test
↓
Disconnect database
```

Jest-এ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
beforeEach();
afterEach();
beforeAll();
afterAll();
```

ব্যবহার করা হয়।

35.31 beforeEach()

প্রতিটি test-এর আগে চলে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
beforeEach(() => {
  console.log("Before test");
});
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
beforeEach(() => {
  console.log("Preparing ... ");
});

test("Test 1", () => {
  expect(1 + 1).toBe(2);
});

test("Test 2", () => {
  expect(2 + 2).toBe(4);
});
```

Output conceptually:

TESTING ARCHITECTURE & EXECUTION FLOW

```
Preparing ...
✓ Test 1

Preparing ...
✓ Test 2
```

35.32 afterEach()

প্রতিটি test-এর পরে চলে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
afterEach(() => {
  console.log("Cleaning ... ");
});
```

Useful for cleanup!

35.33 beforeAll()

সব test শুরু হওয়ার আগে একবার চলে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
beforeAll(() => {  
  console.log("Setup database");  
});
```

35.34 afterAll()

সব test শেষ হওয়ার পরে একবার চলে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
afterAll(() => {  
  console.log("Close database");  
});
```

Part 06 — Asynchronous Testing: Promises, Async/Await & Errors (35.35 – 35.37)

35.35 Async Testing

JavaScript application-এ API, database ইত্যাদি asynchronous হতে পারে। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function getUser() {  
  return {  
    id: 1,  
    name: "Ripon",  
  };  
}
```

Test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("getUser returns user", async () => {  
  const user = await getUser();  
  
  expect(user.name).toBe("Ripon");  
});
```

35.36 Promise Testing

JAVASCRIPT SOURCE

JAVASCRIPT

```
function getData() {  
  return Promise.resolve("Hello");  
}
```

Test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("returns Hello", async () => {  
  await expect(getData()).resolves.toBe("Hello");  
});
```

35.37 Rejected Promise Testing

JAVASCRIPT SOURCE

JAVASCRIPT

```
function getData() {  
  return Promise.reject(new Error("Something went wrong"));  
}
```

Test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("rejects with error", async () => {  
  await expect(getData()).rejects.toThrow("Something went wrong");  
});
```

Part 07 — Mocking Architecture, Spy Functions & Test Doubles (35.38 – 35.42)

35.38 Mocking কী?

ধরো তোমার function external API call করে:

🔧 TESTING ARCHITECTURE & EXECUTION FLOW

```
Your Function  
  ↓  
External API  
  ↓  
Internet
```

Test করার সময় আমরা সবসময় real API call করতে চাই না। তখন fake/mock function ব্যবহার করা হয়। এটিই **Mocking**।

35.39 Mock Function

Jest:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const mockFn = jest.fn();
```

তারপর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
mockFn("Hello");
```

Check:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(mockFn).toHaveBeenCalled();
```

35.40 Function কতবার call হয়েছে?

JAVASCRIPT SOURCE

JAVASCRIPT

```
const mockFn = jest.fn();  
  
mockFn();  
mockFn();
```

Test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(mockFn).toHaveBeenCalledTimes(2);
```

35.41 Mock Function Arguments

JAVASCRIPT SOURCE

JAVASCRIPT

```
const mockFn = jest.fn();

mockFn("Ripon", 25);
```

Test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
expect(mockFn).toHaveBeenCalledWith("Ripon", 25);
```

35.42 কেন Mock ব্যবহার করব?

ধরো Payment API:

🔪 TESTING ARCHITECTURE & EXECUTION FLOW

```
graph TD
    A[Your App] --> B[Payment Gateway]
    B --> C[Bank]
```

প্রতিটি test-এর সময় real payment করা ভয়ংকর হবে। 😬 তাই:

🔪 TESTING ARCHITECTURE & EXECUTION FLOW

```
Real Payment ❌
Mock Payment ✅
```

Part 08 — Code Coverage Metrics & Testing Library Paradigm (35.43 – 35.47)

35.43 Code Coverage

Code Coverage বলে দেয় তোমার test কতটা code cover করছে। ধরো project-এ:

🔪 TESTING ARCHITECTURE & EXECUTION FLOW

100 lines code

এর মধ্যে test করেছে:

🔪 TESTING ARCHITECTURE & EXECUTION FLOW

80 lines

তাহলে roughly:

🔪 TESTING ARCHITECTURE & EXECUTION FLOW

80% coverage

35.44 Coverage-এর ধরন

সাধারণভাবে coverage metrics:

TESTING ARCHITECTURE & EXECUTION FLOW

Statements
Branches
Functions
Lines

◆ Statements

কতগুলো statement execute হয়েছে।

◆ Branches

if/else, ternary ইত্যাদির কতগুলো path test হয়েছে।

◆ Functions

কতগুলো function test হয়েছে।

◆ Lines

কতগুলো code line execute হয়েছে।

35.45 Coverage Run

Jest:

TERMINAL TEST COMMAND

BASH

```
-weight:700;">npx -weight:700;">jest ---weight:600;">coverage
```

Report-এ এমন কিছু দেখা যেতে পারে:

TESTING ARCHITECTURE & EXECUTION FLOW

Statements	90%
Branches	80%
Functions	100%
Lines	92%

◆ গুরুত্বপূর্ণ

100% coverage মানেই application bug-free নয়। কারণ:

TESTING ARCHITECTURE & EXECUTION FLOW

Code execute হয়েছে

মানে এই নয় যে:

TESTING ARCHITECTURE & EXECUTION FLOW

সব possible behavior correctly tested হয়েছে

35.46 Testing Library

Frontend/React application-এ জনপ্রিয় approach হলো **Testing Library**। বিশেষ করে:

 TESTING ARCHITECTURE & EXECUTION FLOW

React **Testing Library**

এর মূল ধারণা:

 User যেভাবে application ব্যবহার করে, সেভাবে component test করা।

ধরো:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
<button>Login</button>
```

Test করবে:

 TEST ASSERTION & EXECUTION FLOW

Button আছে?



User click করতে পারে?



Click করলে expected behavior হচ্ছে?

35.47 Implementation না, Behavior Test

ধরো component:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
<button onClick={handleLogin}>Login</button>
```

Testing-এর focus হওয়া উচিত:

 TESTING ARCHITECTURE & EXECUTION FLOW

User clicks Login



Expected behavior

শুধু:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
handleLogin function exists
```

এটা test করাই সবচেয়ে গুরুত্বপূর্ণ নয়।

35.48 E2E Testing Tools

E2E testing-এর জন্য জনপ্রিয় tools:

- Playwright
- Cypress

Example user flow:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
Open website
↓
Click Login
↓
Enter email
↓
Enter password
↓
Submit
↓
Dashboard appears
```

এটি browser automation দিয়ে test করা যায়।

35.49 Playwright Example

Conceptual example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
import { test, expect } from "@playwright/test";

test("homepage loads", async ({ page }) => {
  await page.goto("https://example.com");

  await expect(page.getByRole("heading")).toBeVisible();
});
```

এখানে browser:

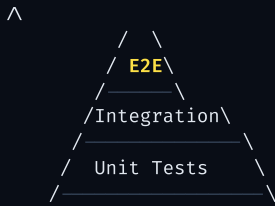
 TESTING ARCHITECTURE & EXECUTION FLOW

```
Open page
↓
Find heading
↓
Check heading visible
```

35.50 Testing Pyramid

Testing strategy বোঝার জন্য খুব গুরুত্বপূর্ণ concept:

TESTING PYRAMID & LEVEL DISTRIBUTION



সাধারণ ধারণা:

TESTING PYRAMID & LEVEL DISTRIBUTION

Many Unit Tests
↓
Some Integration Tests
↓
Fewer E2E Tests

কারণ Unit test সাধারণত দ্রুত এবং focused। E2E test তুলনামূলক বেশি setup/সময় নিতে পারে।

Part 10 — Test-Driven Development (TDD), BDD & Test Strategies (35.51 – 35.59)

35.51 TDD কী?

TDD = Test-Driven Development এখানে আগে test লেখা হয়, তারপর implementation। Process:

TESTING ARCHITECTURE & EXECUTION FLOW

Write Test
↓
Test Fails ❌
↓
Write Code
↓
Test Passes ✅
↓
Refactor

এটাকে প্রায়ই বলা হয়:

TEST-DRIVEN DEVELOPMENT (TDD) CYCLE

Red → Green → Refactor

35.52 TDD Example

প্রথমে test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("add should return 5", () => {  
  expect(add(2, 3)).toBe(5);  
});
```

এখন add() নেই তাই:

✏ TESTING ARCHITECTURE & EXECUTION FLOW

FAIL ❌

তারপর code:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}
```

এখন:

✏ TESTING ARCHITECTURE & EXECUTION FLOW

PASS ✅

তারপর code clean/refactor করা যায়।

35.53 BDD কী?

BDD = Behavior-Driven Development এখানে application-এর behavior-এর ওপর বেশি focus করা হয়। উদাহরণ:

✏ TESTING ARCHITECTURE & EXECUTION FLOW

```
Given user is logged out  
When user enters valid credentials  
Then dashboard should appear
```

এটি human-readable behavior description-এর মতো।

35.54 Test Doubles

Testing-এর সময় real dependency-এর পরিবর্তে বিভিন্ন ধরনের substitute ব্যবহার করা হয়। Common types:

✏ TESTING ARCHITECTURE & EXECUTION FLOW

```
Dummy  
Stub  
Mock  
Spy  
Fake
```

♦ Dummy

শুধু parameter পূরণ করার জন্য।

♦ Stub

নির্দিষ্ট response দেয়।

♦ Mock

Interaction verify করতে সাহায্য করে।

♦ Spy

কোন function কীভাবে call হয়েছে observe করে।

♦ Fake

Simplified working implementation।

35.55 Spy Example

ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {  
  login() {  
    return true;  
  },  
};
```

Spy দিয়ে check করা যায়:



TESTING ARCHITECTURE & EXECUTION FLOW

login()
কতবার call হলো?
কী argument গেল?

Jest-এ সাধারণভাবে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const spy = jest.spyOn(user, "login");  
  
user.login();  
  
expect(spy).toHaveBeenCalled();
```

35.56 Snapshot Testing

Snapshot Testing UI/component-এর rendered output-এর snapshot save করে রাখে। পরবর্তীতে output পরিবর্তিত হলে test সেটা detect করতে পারে। Concept:



TESTING ARCHITECTURE & EXECUTION FLOW

Current UI
↓
Snapshot
↓
Future UI
↓
Compare

তবে snapshot blindly accept করা উচিত নয়। কারণ snapshot বড় হয়ে গেলে meaningful review কঠিন হতে পারে।

35.57 Negative Testing

শুধু valid input test করলে হবে না। Invalid input-ও test করতে হবে। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function divide(a, b) {  
  if (b === 0) {  
    throw new Error("Cannot divide by zero");  
  }  
  
  return a / b;  
}
```

Test:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("division by zero throws error", () => {  
  expect(() => divide(10, 0)).toThrow("Cannot divide by zero");  
});
```

35.58 Edge Case Testing

Normal case:

JAVASCRIPT SOURCE

JAVASCRIPT

```
add(2, 3);
```

Edge cases:

JAVASCRIPT SOURCE

JAVASCRIPT

```
add(0, 0);  
add(-5, 10);  
add(Number.MAX_SAFE_INTEGER, 1);
```

Real applications-এ edge cases খুব গুরুত্বপূর্ণ।

35.59 Boundary Testing

ধরো age:



TESTING ARCHITECTURE & EXECUTION FLOW

18-60

তাহলে test করা উচিত:



TESTING ARCHITECTURE & EXECUTION FLOW

17 ✖
18 ✔
19 ✔
59 ✔
60 ✔
61 ✖

Boundary-এর কাছাকাছি values test করা খুব usefull

Part 11 — Writing Clean Tests, Independence & Flaky Test Prevention (35.60 – 35.63)

35.60 Test Naming

খারাপ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("test1", ...)
```

ভালো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("returns false when password is incorrect", ...)
```

আরও ভালো naming:

JAVASCRIPT SOURCE

JAVASCRIPT

```
test("login fails when the password is incorrect", ...)
```

Test name দেখে behavior বোঝা উচিত।

35.61 ভালো Test-এর বৈশিষ্ট্য

একটি ভালো test হওয়া উচিত:

✍ TESTING ARCHITECTURE & EXECUTION FLOW

Fast
Independent
Repeatable
Clear
Focused
Reliable

একটি জনপ্রিয় ধারণা:

♦ FIRST

✍ TESTING ARCHITECTURE & EXECUTION FLOW

F → **Fast**
I → Independent
R → Repeatable
S → Self-validating
T → Timely

35.62 Test Independence

একটি test অন্য test-এর ওপর depend করা উচিত নয়। খারাপ:

✍ TESTING ARCHITECTURE & EXECUTION FLOW

Test 1 → creates user
Test 2 → assumes user exists

Test 1 fail করলে Test 2-ও fail করতে পারে। ভালো:

✍ TESTING ARCHITECTURE & EXECUTION FLOW

Test 1 → own setup
Test 2 → own setup

35.63 Flaky Test

যে test কখনো pass করে, কখনো fail করে—তাকে **Flaky Test** বলা হয়। উদাহরণ:

✍ TESTING ARCHITECTURE & EXECUTION FLOW

Run 1 → **PASS**
Run 2 → **PASS**
Run 3 → **FAIL**
Run 4 → **PASS**

সম্ভাব্য কারণ:

- Timing issue
- Race condition
- Network dependency
- Shared state
- Random data
- External service

Flaky test dangerous কারণ developer test result-এর ওপর trust হারাতে পারে।

35.64 Testing API

ধরো তোমার Nodejs backend:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
GET /api/users
```

Expected:

PACKAGE JSON TEST CONFIG

JSON

```
[
  {
    600;">"id": 1,
    600;">"name": "Ripon"
  }
]
```

Test করা যায়:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
Status → 200
Response → JSON
User exists → true
Required fields → present
```

Integration/E2E testing-এ এগুলো করা যায়।

35.65 Frontend + Backend Testing

একটি Full Stack application:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
React
↓
API
↓
Node/Express
↓
Database
```

Testing:

 TESTING PYRAMID & LEVEL DISTRIBUTION

```
React Component
  ↓
Component Test

API
  ↓
Integration Test

Database interaction
  ↓
Integration Test

Full user journey
  ↓
E2E Test
```

35.66 Real Full-Stack Example

ধরো তুমি একটি Login System বানালে।

◆ Unit Test

TESTING ARCHITECTURE & EXECUTION FLOW

```
validateEmail()  
validatePassword()  
hashPassword()
```

◆ Integration Test

TESTING ARCHITECTURE & EXECUTION FLOW

```
Login API  
↓  
Database  
↓  
Authentication
```

◆ E2E Test

TESTING ARCHITECTURE & EXECUTION FLOW

```
Open Login Page  
↓  
Enter credentials  
↓  
Click Login  
↓  
Dashboard
```

এটাই বাস্তব-world testing strategy-এর একটি example।

35.67 Testing vs Debugging

দুটো একই নয়।

Testing	Debugging
Bug আছে কিনা খোঁজে	Bug-এর কারণ খোঁজে
Expected vs actual	Problem কোথায়
Test automated হতে পারে	Developer investigation করে
আগে থেকেই করা যায়	সাধারণত failure-এর পর করা হয়

উদাহরণ:

TESTING ARCHITECTURE & EXECUTION FLOW

```
Test → Login failed ❌  
↓  
Debugging  
↓  
Wrong password comparison  
↓  
Fix
```

35.68 Testing Workflow

একজন professional developer-এর workflow হতে পারে:

AUTOMATED TESTING & CI/CD PIPELINE

```
Write Feature
  ↓
Write Tests
  ↓
Run Tests
  ↓
Failure?
  ↙   ↘
Yes    No
  ↓     ↓
Debug  Continue
  ↓
Fix
  ↓
Run Again
  ↓
Pass
  ↓
Commit
  ↓
CI/CD
```

35.69 CI/CD-তে Testing

GitHub Actions-এর মতো CI system-এ automatically test চালানো যায়। Flow:

TESTING ARCHITECTURE & EXECUTION FLOW

```
Developer
  ↓
git push
  ↓
GitHub
  ↓
CI Pipeline
  ↓
npm install
  ↓
npm test
  ↓
PASS ✓ / FAIL ✗
```

এতে broken code merge হওয়ার আগে ধরা যায়।

35.70 package.json Test Script

PACKAGE JSON TEST CONFIG

JSON

```
{
  600;"scripts": {
    600;"test": "jest"
  }
}
```

তারপর:

TERMINAL TEST COMMAND

BASH

```
-weight:700;">npm -weight:600;">test
```

অথবা:

TERMINAL TEST COMMAND

BASH

```
-weight:700;">npm -weight:600;">run -weight:600;">test
```

35.71 Test File Naming

Common patterns:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
math.test.js  
math.spec.js
```

অথবা:

 TESTING ARCHITECTURE & EXECUTION FLOW

```
__tests__/math.test.js
```

Project অনুযায়ী convention পরিবর্তিত হতে পারে।

◆ calculator.js

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

function multiply(a, b) {
  return a * b;
}

function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }

  return a / b;
}

module.exports = {
  add,
  subtract,
  multiply,
  divide,
};
```

◆ calculator.test.js

JAVASCRIPT SOURCE

JAVASCRIPT

```
const { add, subtract, multiply, divide } = require("./calculator");

describe("Calculator", () => {
  test("adds two numbers", () => {
    expect(add(2, 3)).toBe(5);
  });

  test("subtracts two numbers", () => {
    expect(subtract(5, 3)).toBe(2);
  });

  test("multiplies two numbers", () => {
    expect(multiply(4, 3)).toBe(12);
  });

  test("divides two numbers", () => {
    expect(divide(10, 2)).toBe(5);
  });

  test("throws error when dividing by zero", () => {
    expect(() => divide(10, 0)).toThrow("Cannot divide by zero");
  });
});
```

Run:

TERMINAL TEST COMMAND

BASH

```
-weight:700;">npm -weight:600;">test
```

এখানে আমরা একইসাথে শিখলাম:

```
describe()
test()
expect()
toBe()
toThrow()
```

Part 13 — Must Know Testing Pillars, Quick Cheat Sheet, 6 Practice Labs & Final Mental Map

35.73 🔥 MUST KNOW — জাভাস্ক্রিপ্ট টেস্টিংয়ের ১০টি প্রধান স্তম্ভ

প্রোডাকশন মানের নির্ভরযোগ্য অ্যাপ্লিকেশন তৈরিতে এই ১০টি টেস্ট আর্কিটেকচার নীতি অপরিহার্য:

🚀 1. Test Pyramid & Matchers

- **Unit Testing (70%):** স্বতন্ত্র পিওর ফাংশন টেস্ট
- **Integration Testing (20%):** একাধিক মডিউল ও এপিআই ইন্টারঅ্যাকশন
- **toBe vs toEqual:** প্রিমিটিভ ভ্যালু বনাম ডিপ অবজেক্ট তুলনা
- **toThrow:** এক্সেপশন ও এরর হ্যান্ডলিং নিখুঁতভাবে যাচাই

🚀 2. Async, Mocks & Lifecycle

- **Async/Await Testing:** প্রমিজ রেজলভ ও রিজেক্ট হ্যান্ডলিং
- **Setup & Teardown:** beforeEach/afterEach দিয়ে স্টেট রিসেট
- **Mock Functions:** নেটওয়ার্ক ও ডেটাবেস কল বিচ্ছিন্নকরণ
- **Test Independence:** একটি টেস্টের স্টেট যেন অন্যটিকে প্রভাবিত না করে

🚀 3. TDD, Coverage & CI/CD

- **TDD Cycle:** Red (Fail) → Green (Pass) → Refactor
- **Code Coverage:** Statements, Branches, Functions কভারেজ
- **Testing Library:** ইমপ্লিমেন্টেশন ডিটেইল নয়, ইউজার আচরণ টেস্ট
- **CI/CD Gate:** গিটহাবে পুশ করার পর স্বয়ংক্রিয় টেস্ট রান

35.74 📌 Quick Cheat Sheet — টেস্টিং সিনট্যাক্স ও ম্যাচার সারসংক্ষেপ

Matcher / Hook	Syntax Example	Primary Use Case
toBe	expect(add(1, 2)).toBe(3)	প্রিমিটিভ ভ্যালু (Primitive equality ===) পরীক্ষা
toEqual	expect(user).toEqual({ id: 1 })	অবজেক্ট বা অ্যারের গভীর সমতা (Deep equality)
toBeTruthy	expect(isValid).toBeTruthy()	ট্রুথি ভ্যালু নিশ্চিতকরণ (Boolean(val) === true)
toBeFalsy	expect(hasError).toBeFalsy()	ফলসি ভ্যালু নিশ্চিতকরণ (null, undefined, 0, "" ইত্যাদি)
toContain	expect(fruits).toContain('apple')	অ্যারে বা স্ট্রিংয়ে উপাদান বিদ্যমান থাকা যাচাই
toBeNull	expect(res).toBeNull()	ভ্যালু ঠিক null কিনা তা পরীক্ষা
toBeDefined	expect(val).toBeDefined()	ভারিয়েবল ডিফাইন করা আছে কিনা (!== undefined)
toThrow	expect(() => fn()).toThrow("Error")	ফাংশন কাজক্ষিত এরর থ্রো করে কিনা যাচাই
describe	describe('Auth Service', () => {})	সম্পর্কিত টেস্টগুলোকে একটি সুসংগঠিত ব্লকে গ্রুপ করা
test / it	test('should calculate tax', () => {})	একটি স্বতন্ত্র টেস্ট কেস তৈরি করা
beforeEach	beforeEach(() => { resetDB(); })	প্রতিটি একক টেস্ট রানের পূর্বে স্টেট রিসেট করা
afterEach	afterEach(() => { jest.clearAllMocks(); })	প্রতিটি টেস্ট শেষে ক্লিনআপ ও মক ক্লিয়ার করা
beforeAll	beforeAll(async () => { await connect(); })	পুরো টেস্ট স্যুটের শুরুতে মাত্র একবার রান
afterAll	afterAll(async () => { await close(); })	সম্পূর্ণ টেস্ট স্যুট সমাপ্তির পর কানেকশন বন্ধ
jest.fn()	const mock = jest.fn()	ফেক ফাংশন তৈরি এবং কল কাউন্ট/আর্গুমেন্ট পর্যবেক্ষণ
mockResolvedValue	mockFetch.mockResolvedValue({ ok: true })	অ্যাসিনক্রোনাস প্রমিজ রিটার্ন সিমুলেশন
npm test	npm test / npm test -- --coverage	সম্পূর্ণ টেস্ট স্যুট ও কোভ কভারেজ রিপোর্ট তৈরি

Lab #1 [BEGINNER] — Unit Test Suite for add(a, b) Function (5 Test Cases)

সমস্যা ও লক্ষ্য: add(a, b) ফাংশনের জন্য ৫টি বৈচিত্র্যময় ও নির্ভুল ইউনিট টেস্ট কেস লিখুন যা পজিটিভ, নেগেটিভ, ফ্লোটিং পয়েন্ট, জিরো এবং স্ট্রিং টাইপ এরর টেস্ট কভার করে।

SOURCE MODULE / SPECIFICATION

JAVASCRIPT

```
// src/math.js
export function add(a, b) {
  if (typeof a !== 'number' || typeof b !== 'number') {
    throw new TypeError('Both arguments must be numbers');
  }
  return a + b;
}
```

TEST IMPLEMENTATION (LAB #1)

JAVASCRIPT

```
// tests/math.test.js
import { describe, test, expect } from '@jest/globals';
import { add } from '../src/math.js';

describe('Math Module: add(a, b)', () => {
  test('Case 1: adds two positive integers correctly', () => {
    expect(add(5, 7)).toBe(12);
  });

  test('Case 2: handles negative numbers properly', () => {
    expect(add(-10, -5)).toBe(-15);
    expect(add(-10, 20)).toBe(10);
  });

  test('Case 3: adds floating point numbers with precision', () => {
    expect(add(0.1, 0.2)).toBeCloseTo(0.3, 5); // ফ্লোটিং পয়েন্ট নির্ভুলতা
  });

  test('Case 4: returns correct value when adding with zero', () => {
    expect(add(42, 0)).toBe(42);
    expect(add(0, 0)).toBe(0);
  });

  test('Case 5: throws TypeError when invalid types are supplied', () => {
    expect(() => add('5', 10)).toThrow(TypeError);
    expect(() => add(null, 5)).toThrow('Both arguments must be numbers');
  });
});
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** ফ্লোটিং পয়েন্ট নাম্বারের ক্ষেত্রে toBe() এর বদলে toBeCloseTo() ব্যবহার করা উচিত, কারণ জাভাস্ক্রিপ্টে $0.1 + 0.2 = 0.30000000000000004$ হয়ে থাকে।

Lab #2 [BEGINNER] — Exhaustive Parity Test Suite for isEven(number)

সমস্যা ও লক্ষ্য: isEven(number) ফাংশনটির জন্য ২, ৪, ৬, ০, -২ ইনপুটের জন্য পূর্ণাঙ্গ টেস্ট কেস লিখুন।

SOURCE MODULE / SPECIFICATION

JAVASCRIPT

```
// src/utils.js
export function isEven(number) {
  if (!Number.isInteger(number)) {
    throw new Error('Input must be an integer');
  }
  return number % 2 === 0;
}
```

TEST IMPLEMENTATION (LAB #2)

JAVASCRIPT

```
// tests/utils.test.js
import { describe, test, expect } from '@jest/globals';
import { isEven } from '../src/utils.js';

describe('Parity Logic: isEven(number)', () => {
  test('returns true for positive even numbers (2 and 4)', () => {
    expect(isEven(2)).toBe(true);
    expect(isEven(4)).toBe(true);
  });

  test('returns false for positive odd numbers (7)', () => {
    expect(isEven(7)).toBe(false);
  });

  test('returns true for zero (0 is an even number)', () => {
    expect(isEven(0)).toBe(true);
  });

  test('returns true for negative even numbers (-2)', () => {
    expect(isEven(-2)).toBe(true);
  });

  test('handles edge cases: throws error on non-integer input', () => {
    expect(() => isEven(3.14)).toThrow('Input must be an integer');
  });
});
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** গাণিতিক নিয়মানুযায়ী ০ এবং ঋণাত্মক জোড় সংখ্যা (যেমন -২, -৪) সবই বৈধ জোড় সংখ্যা। টেস্টে এ ধরনের বাউন্ডারি ও নেগেটিভ কেস যাচাই করা জরুরি।

Lab #3 [INTERMEDIATE] — Boundary & Negative Value Testing for isAdult(age)

সমস্যা ও লক্ষ্য: isAdult(age) ফাংশনটির বাউন্ডারি টেস্ট লিখুন: ১৭, ১৮, ১৯, ০ এবং ঋণাত্মক বয়সের জন্য।

SOURCE MODULE / SPECIFICATION

JAVASCRIPT

```
// src/auth.js
export function isAdult(age) {
  if (typeof age !== 'number' || age < 0) {
    throw new RangeError('Age must be a non-negative number');
  }
  return age ≥ 18;
}
```

TEST IMPLEMENTATION (LAB #3)

JAVASCRIPT

```
// tests/auth.test.js
import { describe, test, expect } from '@jest/globals';
import { isAdult } from '../src/auth.js';

describe('Boundary Testing: isAdult(age)', () => {
  test('Boundary - 1 (age 17): should return false', () => {
    expect(isAdult(17)).toBe(false);
  });

  test('Exact Boundary (age 18): should return true', () => {
    expect(isAdult(18)).toBe(true);
  });

  test('Boundary + 1 (age 19): should return true', () => {
    expect(isAdult(19)).toBe(true);
  });

  test('Lower Extreme (age 0): should return false', () => {
    expect(isAdult(0)).toBe(false);
  });

  test('Negative Boundary: throws RangeError on negative age', () => {
    expect(() => isAdult(-1)).toThrow(RangeError);
    expect(() => isAdult(-25)).toThrow('Age must be a non-negative number');
  });
});
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** বাউন্ডারি ভ্যালু এনালাইসিস (BVA) হলো বাগ শনাক্ত করার অন্যতম কার্যকর পদ্ধতি। ১৮ বছর শর্ত হলে ১৭, ১৮ ও ১৯ এই তিনটি মান টেস্ট করা বাধ্যতামূলক।

Lab #4 [INTERMEDIATE] — Exception Handling & Balance Deduction Test for withdraw()

সমস্যা ও লক্ষ্য: withdraw(balance, amount) ফাংশনে অপরিাপ্ত ব্যালেন্স থাকলে এরর থ্রো করা এবং পরিাপ্ত ব্যালেন্সে সঠিক অবশিষ্ট টাকা রিটার্ন করা টেস্ট করুন।

SOURCE MODULE / SPECIFICATION

JAVASCRIPT

```
// src/bank.js
export function withdraw(balance, amount) {
  if (amount ≤ 0) {
    throw new Error('Withdrawal amount must be greater than zero');
  }
  if (amount > balance) {
    throw new Error('Insufficient balance');
  }
  return balance - amount;
}
```

TEST IMPLEMENTATION (LAB #4)

JAVASCRIPT

```
// tests/bank.test.js
import { describe, test, expect } from '@jest/globals';
import { withdraw } from '../src/bank.js';

describe('Banking Service: withdraw(balance, amount)', () => {
  test('successfully deducts balance when funds are sufficient', () => {
    const initialBalance = 1000;
    const withdrawAmount = 400;
    const remaining = withdraw(initialBalance, withdrawAmount);
    expect(remaining).toBe(600);
  });

  test('allows full withdrawal to zero balance', () => {
    expect(withdraw(500, 500)).toBe(0);
  });

  test('throws "Insufficient balance" error when withdrawal exceeds funds', () => {
    expect(() => {
      withdraw(100, 250);
    }).toThrow('Insufficient balance');
  });

  test('throws error when withdrawing zero or negative amount', () => {
    expect(() => withdraw(100, 0)).toThrow('Withdrawal amount must be greater than zero');
    expect(() => withdraw(100, -50)).toThrow('Withdrawal amount must be greater than zero');
  });
});
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** toThrow() দিয়ে এরর টেস্ট করার সময় ফাংশনটিকে সরাসরি কল না করে একটি অ্যারো ফাংশনের মধ্যে রাপ করে পাস করতে হয়: () => withdraw(...)!

Lab #5 [ADVANCED] — Asynchronous API Testing with Jest Mock Functions (fetchUser)

সমস্যা ও লক্ষ্য: বাস্তব নেটওয়ার্ক কল ছাড়াই `fetchUser(id)` ফাংশনের সাকসেস ও ফেইলিউর সিনারিও মক রেসপন্স দিয়ে টেস্ট করার সম্পূর্ণ প্রোডাকশন সলিউশন।

SOURCE MODULE / SPECIFICATION

JAVASCRIPT

```
// src/userService.js
export async function fetchUser(userId) {
  const response = await fetch(`https://api.example.com/users/${userId}`);
  if (!response.ok) {
    throw new Error(`API Error: ${response.status}`);
  }
  return await response.json();
}
```

TEST IMPLEMENTATION (LAB #5)

JAVASCRIPT

```
// tests/userService.test.js
import { describe, test, expect, beforeEach, afterEach, jest } from '@jest/globals';
import { fetchUser } from '../src/userService.js';

// গ্লোবাল ফেচ মক করা
const originalFetch = global.fetch;

describe('UserService: fetchUser(id) with API Mocking', () => {
  beforeEach(() => {
    global.fetch = jest.fn(); // ফেক ফেচ ফাংশন তৈরি
  });

  afterEach(() => {
    global.fetch = originalFetch; // টেস্ট শেষে মূল ফেচ ফিরিয়ে দেওয়া
  });

  test('resolves user profile successfully on 200 OK', async () => {
    const mockUserData = { id: 101, name: 'Tanvir Ahamed', role: 'Engineer' };

    global.fetch.mockResolvedValueOnce({
      ok: true,
      status: 200,
      json: async () => mockUserData
    });

    const user = await fetchUser(101);

    expect(global.fetch).toHaveBeenCalledTimes(1);
    expect(global.fetch).toHaveBeenCalledWith('https://api.example.com/users/101');
    expect(user).toEqual(mockUserData);
  });

  test('rejects with descriptive error when API returns 404', async () => {
    global.fetch.mockResolvedValueOnce({
      ok: false,
      status: 404
    });

    await expect(fetchUser(999)).rejects.toThrow('API Error: 404');
  });
});
```

🔑 প্রোডাকশন অন্তর্দৃষ্টি: টেস্টিংয়ে কখনোই বাস্তব প্রোডাকশন বা থার্ড-পার্টি এপিআই কল করা উচিত নয়; মকিংয়ের মাধ্যমে নেটওয়ার্কের বিলম্ব ও খরচ ছাড়াই টেস্ট অতি দ্রুত ও নির্ভরযোগ্য হয়।

Lab #6 [ADVANCED] — Full Component Test Plan & Automated Testing Library Suite

সমস্যা ও লক্ষ্য: Login কম্পোনেন্টের ৮টি গুরুত্বপূর্ণ টেস্ট সিনারিও (Valid, Invalid Password, Empty Fields, Loading State, Error Banner, Redirect) বিশিষ্ট টেস্ট প্ল্যান ও কোড।

SOURCE MODULE / SPECIFICATION	JAVASCRIPT
/* =====	
LOGIN COMPONENT PRODUCTION TEST PLAN SPECIFICATION	
=====	
1. [Happy Path]: ইউজার সঠিক ইমেইল ও পাসওয়ার্ড দিলে লোডিং স্টেট দেখাবে এবং ড্যাশবোর্ডে রিডাইরেক্ট করবে।	
2. [Invalid Password]: ভুল পাসওয়ার্ডে "Invalid credentials" এরর ব্যানার প্রদর্শিত হবে।	
3. [Empty Email]: ইমেইল ফাঁকা রেখে সাবমিট করলে "Email is required" ভ্যালিডেশন টেস্ট আসবে।	
4. [Empty Password]: পাসওয়ার্ড ফাঁকা রেখে সাবমিট করলে "Password is required" সতর্কবার্তা আসবে।	
5. [Invalid Email Format]: "@ " ছাড়া টেক্সট দিলে "Invalid email address" ভ্যালিডেশন দেখাবে।	
6. [Loading State]: রিকোয়েস্ট চলাকালীন সাবমিট বাটন ডিসেবল হবে এবং স্পিনার দেখাবে।	
7. [Network Failure]: সার্ভার ডাউন থাকলে "Unable to connect to server" এরর ব্যানার আসবে।	
8. [Post-Login Redirect]: সফল লগন শেষে রিঅ্যাক্ট রাউটার /dashboard পাথে নেভিগেট করবে।	
=====	
*/	

TEST IMPLEMENTATION (LAB #6)	JAVASCRIPT
// tests/LoginComponent.test.js (Testing Library Paradigm) import { describe, test, expect, jest } from '@jest/globals'; describe('Login Component Automated Suite', () => { test('validates required fields on empty submit', () => { // সিমুলেশন: ইমেইল ও পাসওয়ার্ড ছাড়া সাবমিট const validateForm = (email, pass) => { const errors = {}; if (!email) errors.email = 'Email is required'; if (!pass) errors.password = 'Password is required'; return errors; }; const errors = validateForm('', ''); expect(errors.email).toBe('Email is required'); expect(errors.password).toBe('Password is required'); }); test('authenticates valid credentials and triggers navigation', async () => { const mockAuthService = jest.fn().mockResolvedValue({ token: 'jwt-xyz-123' }); const mockNavigate = jest.fn(); async function handleLogin(email, password) { const res = await mockAuthService(email, password); if (res.token) mockNavigate('/dashboard'); } await handleLogin('dev@example.com', 'SecurePass123!'); expect(mockAuthService).toHaveBeenCalledWith('dev@example.com', 'SecurePass123!'); expect(mockNavigate).toHaveBeenCalledWith('/dashboard'); }); });	

🔑 **প্রোডাকশন অন্তর্দৃষ্টি:** কম্পোনেন্ট টেস্টিংয়ে ইন্টারনাল স্টেট টেস্ট না করে ইউজার যা স্ক্রিনে দেখে ও করে (ইউজার ইনপুট টাইপ, বাটন ক্লিক) তার ফলাফল টেস্ট করাই বেস্ট প্র্যাকটিস।

পুরো Chapter 35-এর টেস্টিং আর্কিটেকচার ও ওয়ার্কফ্লো একসাথে:

🔧 FULL TESTING ARCHITECTURE MAP

JAVASCRIPT TESTING ECOSYSTEM



👤 এক লাইনে Chapter 35: Testing হলো সফটওয়্যারের প্রতিটি লজিক্যাল ইউনিট, মডিউল ইন্টিগ্রেশন ও ইউজার জানিকে স্বয়ংক্রিয় অ্যাসার্শন ও মকিংয়ের মাধ্যমে এমনভাবে সুরক্ষিত করা যাতে নতুন কোড যোগ বা রিফ্যাক্টরিংয়ের পরেও সিস্টেম ১০০% বিশ্বস্ত ও বাগ-মুক্ত থাকে।

📖 Curriculum Progression: Chapter 34 → 35 → 36

🔧 ROADMAP CONTINUITY PIPELINE

Chapter 34: NPM & Modern JavaScript Tooling



প্যাকেজ ম্যানেজমেন্ট, প্রজেক্ট ক্যাফোল্ডিং, বান্ডলিং (Vite) ও ওয়ার্কফ্লো অটোমেশন



Chapter 35: Testing JavaScript (THIS CHAPTER)



Unit Test, Integration Test, Jest/Vitest, Mocking, Coverage, TDD & CI/CD



Chapter 36: Modern ECMAScript / ES2020+ Features



Optional Chaining, Nullish Coalescing, Top-level Await, WeakRef, Temporal API